Introduction to Programming

Mid Semester LAB Exam Solutions 100 MARKS

ROLL NO. _____

1. Magical Array Split (Easy Level): 25 MINUTES

[30 MARKS]

Arjun is fascinated by number patterns. He defines a special kind of array, called a **Magical Array**, based on the following conditions:

1. The program must read an integer n (size of the array) followed by n integers (array elements).
2. There must exist at least one index in the array such that the sum of elements on the left side equals the sum of elements on the right side.
3. The array must **not** be a palindrome.
4. The *recursive digit sum* of all elements in the array must be a prime number.

Input Format:

- The input will be given in two lines:
  1. First line: An integer n (array size).
  2. Second line: n integers (the array elements).

Output Format:

- First line: `YES` if the array is magical, otherwise `NO`.
- Second line: the recursive digit sum of all elements.

Example 1

Input

```
5
12 7 9 4 21
```

Processing Steps:

- Recursive digit sum $= (1+2) + (7) + (9) + (4) + (2+1) = 26 = 2+6 = 8$ (not prime).
- The array is not a palindrome.
- No split satisfies as LHS sum $\neq$ RHS sum.
- Split at $k = 0$: sum (12), sum $(7+9+4+21) = 41$. Not equal.
- Split at $k = 1$: sum $(12+7) = 19$, sum $(9+4+21) = 34$. Not equal.
- Split at $k = 2$: sum $(12+7+9) = 28$, sum $(4+21) = 25$. Not equal.
- Split at $k = 3$: sum $(12+7+9+4) = 32$, sum (21). Not equal.
- Conditions not met.

Output

```
NO
8
```

Note: The Magical Array Split problem is made based on mixed problems shared from practice-set of recursion.

#	INPUT	OUTPUT	VISIBILITY
1	5 1 2 3 2 1	NO 9	visible
2	4 10 20 30 40	NO 1	visible
3	6 2 3 5 2 1 4	NO 8	visible
4	4 5 1 2 8	YES 7	invisible
5	6 4 9 7 7 9 4	NO 4	invisible
6	5 99 100 55 23 18	NO 7	invisible
7	5 8 8 8 8 8	NO 4	invisible
8	3 11 22 11	NO 8	invisible
9	5 2 4 6 8 10	NO 3	invisible
10	6 2 3 5 4 2 4	YES 2	invisible
11	4 6 4 4 6	NO 2	invisible
12	4 7 3 5 5	YES 2	invisible
13	5 9 1 2 3 5	YES 2	invisible
14	6 4 8 2 6 5 5	NO 3	invisible
15	4 9 1 6 4	YES 2	invisible
16	10 8 9 8 9 8 9 8 9 8 9	NO 4	invisible
17	15 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	NO 6	invisible
18	4 8 2 5 3	NO 9	invisible
19	6 3 4 6 5 3 6	NO 9	invisible
20	11 1 2 3 4 5 6 7 8 9 10 55	YES 2	invisible

Figure 1: Test-Cases for problem-1

```c
#include <stdio.h>

// Recursive function to calculate sum of array elements in [start..end]
int sum(int arr[], int start, int end) {
    if (start > end) return 0;
    return arr[start] + sum(arr, start + 1, end);
}

// Function to calculate digit sum of a number
int digitSum(int num) {
    if (num == 0) return 0;
    return (num % 10) + digitSum(num / 10);
}

// Recursive function to calculate recursive digit sum of an array
int recursiveDigitSum(int arr[], int n) {
    int total = sum(arr, 0, n - 1);
    int digitsum = digitSum(total);
    if (digitsum < 10) return digitsum;
    return digitSum(digitsum);
}

// Recursive function to check if a number is prime
int isPrimeHelper(int num, int divisor) {
    if (divisor * divisor > num) return 1;
    if (num % divisor == 0) return 0;
    return isPrimeHelper(num, divisor + 1);
}
int isPrime(int num) {
    if (num < 2) return 0;
    return isPrimeHelper(num, 2);
```

```c
}

// Recursive function to check palindrome
int isPalindromeRec(int arr[], int left, int right) {
    if (left >= right) return 1;
    if (arr[left] != arr[right]) return 0;
    return isPalindromeRec(arr, left + 1, right - 1);
}

// Recursive function to check split
int canSplitRec(int arr[], int n, int index) {
    if (index >= n - 1) return 0;
    int leftSum = sum(arr, 0, index);
    int rightSum = sum(arr, index + 1, n - 1);
    if (leftSum == rightSum) return 1;
    return canSplitRec(arr, n, index + 1);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int rds = recursiveDigitSum(arr, n);

    int magical = 1;
    if (!canSplitRec(arr, n, 0)) magical = 0;
    if (isPalindromeRec(arr, 0, n - 1)) magical = 0;
    if (!isPrime(rds)) magical = 0;

    if (magical) printf("YES\n");
    else printf("NO\n");
    printf("%d\n", rds);

    return 0;
}
```

2. Jospehus with Chemical Compounds (Moderate - Difficult level): 80 MINUTES

[40 MARKS]

Josephus recently attended a chemistry class where he learned the **first 30 elements of the periodic table** and their **integer atomic weights**. Back home, he tried to teach the same to his friends. Because his friends were not used to writing formulas, they produced chemical formulas full of **parentheses () and brackets []** with multipliers, like `(OH)2` or `[Fe(CN)6]3`. Josephus wanted them to compute the **total molecular weights**, but the formulas became so long and nested that he could **no longer evaluate them manually**. He therefore asks you to write a program to compute the **integer molecular weight** of a given compound automatically.

In other words, for a compound string S, you must compute:

$$\textbf{TotalWeight}(S) = \sum_{\text{each element } E_i} W(E_i)\, n_i + \sum_{\text{each group } G_j} W(G_j)\, m_j$$

where:

1. $W(E_i)$ is the atomic weight of element E_i from the table,
2. n_i is the number immediately after the element (default 1),
3. $W(G_j)$ is the total weight of a group in parentheses or brackets,
4. m_j is the multiplier immediately after the closing bracket (default 1).

Nested groups are evaluated **from the innermost to the outermost**.

Atomic Weights Table (first 30 elements):

#	Symbol	Weight
1	H	1
2	He	4
3	Li	7
4	Be	9
5	B	11
6	C	12
7	N	14
8	O	16
9	F	19
10	Ne	20

#	Symbol	Weight
11	Na	23
12	Mg	24
13	Al	27
14	Si	28
15	P	31
16	S	32
17	Cl	35
18	Ar	40
19	K	39
20	Ca	40

#	Symbol	Weight
21	Sc	45
22	Ti	48
23	V	51
24	Cr	52
25	Mn	55
26	Fe	56
27	Co	59
28	Ni	59
29	Cu	64
30	Zn	65

Input: A single line containing the chemical formula (length ≤ 100).

Output: Print a single integer: the total molecular weight of the given compound.

Constraints:

1. The compound uses only the above 30 elements.
2. The string may contain uppercase A–Z and lowercase a–z for element symbols, digits 0–9 for multiplicities, and parentheses () or square brackets [].
3. Multipliers are positive integers ≤ 99.
4. No invalid syntax will be given; a valid answer is always guaranteed.
5. Use only loops and conditional statements—no external libraries beyond standard C headers.
6. Output must be terminated by the newline character.

Sample Test Case 1:

Input:

```
C12H22O11NaCl
```

Output:

```
400
```

Explanation:

$$\text{TotalWeight} = 12 \cdot 12 + 1 \cdot 22 + 16 \cdot 11 + 23 \cdot 1 + 35 \cdot 1 = 400$$

Sample Test Case 2:

Input:

```
[Fe(CN)6]3
```

Output:

```
636
```

Explanation:

$$\text{TotalWeight}((CN)_6) = 12 \cdot 6 + 14 \cdot 6 = 156, \quad W([Fe(CN)_6]) = 56 + 156 = 212, \quad 212 \cdot 3 = 636$$

Sample Test Case 3:

Input:

```
K6[Ca2(Mg(PO4)3(SiO4)2)2](OH)8Na4Cl6
```

Output:

1738

Explanation Compute innermost groups:

$$(PO_4)_3 = (31 + 16 \cdot 4) \cdot 3 = 285, \quad (SiO_4)_2 = (28 + 16 \cdot 4) \cdot 2 = 184$$

Combine with Mg:

$$24 + 285 + 184 = 493$$

Multiply by Ca_2:

$$40 \cdot 2 + 493 \cdot 2 = 1066$$

Add $(OH)_8$:

$$(16 + 1) \cdot 8 = 136$$

Add K_6, Na_4, Cl_6:

$$39 \cdot 6 = 234, \quad 23 \cdot 4 = 92, \quad 35 \cdot 6 = 210$$

Sum:

$$1066 + 136 + 234 + 92 + 210 = 1738$$

Note: This question is an improvisation of LAB-3 (Problem-3), Revision LAB (Problem-3, Problem-4) and added an extra (), [] parenthesis concept.

#	INPUT	OUTPUT	VISIBILITY
1	C2H4O6	124	visible
2	NO2	46	visible
3	Na2SO4	142	visible
4	NaCl	58	visible
5	H2SO4	98	visible
6	Ca(OH)2	74	visible
7	CaO(Cl)2	126	visible
8	C12H22O11NaCl	400	invisible
9	Ca5(PO4)3(OH)2Na2	565	invisible
10	K4Fe(CN)6	368	invisible
11	C20H25N3O2FeNaCl2	488	invisible
12	CH3CONHC6H4OC2H5	179	invisible
13	C19H29COOH	302	invisible
14	CH3CN	41	invisible
15	Ca3(PO4)2Mg(OH)6Na2	482	invisible
16	K12[Ca3(PO4)2(SO4)3(H2O)5]2	1844	invisible
17	Na4[Co(NiCl4)(Fe(CN)6)]2(OH)6	1134	invisible
18	C24H36N6O12Fe3Na4Cl6	1070	invisible
19	Ca10[Al2(SiO4)5(Fe(CN)6)3](OH)12Na8	1938	invisible
20	Na8[Fe3(PO4)4(CN)12](OH)6Cl4H2O	1304	invisible
21	Na40[Co5(NiCl4)3(Fe(CN)6)6(Mg(PO4)3(SiO4)2)3](OH)30H50	5123	invisible
22	[Fe(CN)6]3	636	visible
23	K6[Ca2(Mg(PO4)3(SiO4)2)2](OH)8Na4Cl6	1738	visible
24	Ca20[Al2(SiO4)5(Fe(CN)6)3](OH)12H8	2162	invisible
25	Na4[Co(ZnCl4)(Fe(CN)6)]2(OH)6	1146	invisible

Figure 2: Test-Cases for problem-2

```c
#include <stdio.h>

#define MAX 200

/* first 30 element symbols */
char symbols[30][3] = {
    "H","He","Li","Be","B","C","N","O","F","Ne",
    "Na","Mg","Al","Si","P","S","Cl","Ar","K","Ca",
    "Sc","Ti","V","Cr","Mn","Fe","Co","Ni","Cu","Zn"
};

/* integer atomic weights */
int weights[30] = {
    1,4,7,9,11,12,14,16,19,20,
    23,24,27,28,31,32,35,40,39,40,
    45,48,51,52,55,56,59,59,64,65
};

/* manual string length */
int mylen(char s[]) {
```

```
    int n=0;
    while(s[n]!='\0') n++;
    return n;
}

/* manual compare */
int mycmp(char a[], char b[]) {
    int i=0;
    while(a[i]!='\0' && b[i]!='\0') {
        if(a[i]!=b[i]) return 0;
        i++;
    }
    if(a[i]=='\0' && b[i]=='\0') return 1;
    return 0;
}

/* uppercase? */
int isUpper(char c) {
    return (c>='A' && c<='Z');
}

/* lowercase? */
int isLower(char c) {
    return (c>='a' && c<='z');
}

/* digit? */
int isDigit(char c) {
    return (c>='0' && c<='9');
}

/* find weight by element symbol */
int getWeight(char sym[3]) {
    int i;
    for (i=0; i<30; i++) {
        if (mycmp(symbols[i], sym)==1) {
            return weights[i];
        }
    }
}

int main() {
    char formula[MAX];
    scanf("%s", formula);

    int len = mylen(formula);
    int i = 0;
    int total = 0;

    /* array to keep track of nested group totals */
    int groupTotals[20];
    int level = -1;

    while (i < len) {
        if (formula[i] == '(' || formula[i] == '[') {
            level++;
            groupTotals[level] = 0;
            i++;
        }
        else if (formula[i] == ')' || formula[i] == ']') {
            i++;
            int num = 0;
            while (i < len && isDigit(formula[i])) {
                num = num*10 + (formula[i]-'0');
                i++;
            }
            if (num==0) num = 1;

            int groupSum = groupTotals[level];
            level--;
```

```
91
92              if (level >= 0) {
93                  groupTotals[level] += groupSum*num;
94              } else {
95                  total += groupSum*num;
96              }
97          }
98          else if (isUpper(formula[i])) {
99              char sym[3];
100             sym[0] = formula[i];
101             sym[1] = '\0';
102             sym[2] = '\0';
103             i++;
104             if (i<len && isLower(formula[i])) {
105                 sym[1] = formula[i];
106                 sym[2] = '\0';
107                 i++;
108             }
109
110             int num=0;
111             while (i<len && isDigit(formula[i])) {
112                 num=num*10+(formula[i]-'0');
113                 i++;
114             }
115             if (num==0) num=1;
116
117             int wt=getWeight(sym)*num;
118
119             if (level>=0)
120                 groupTotals[level]+=wt;
121             else
122                 total+=wt;
123         }
124     }
125
126     while (level>=0) {
127         total+=groupTotals[level];
128         level--;
129     }
130
131     printf("%d\n", total);
132     return 0;
133 }
134 }
```

3. Riya-s Benchmark Mystery (Basic Level): 15 MINUTES

[30 MARKS]

Riya is analysing the performance of different departments in her company. She collects the performance data in the form of a square matrix, where each row represents a department, and each column represents a performance metric.

She is searching for a **special value** with the following property:

1. The value should be the *least in its row* (compared to all other values in the same department).
2. At the same time, the value should be the *greatest in its column* (compared to all other departments for that metric).

Input Format

- The first line contains an integer n (the size of the matrix).
- The following n lines each contain n integers, representing the matrix values.

Output Format

- If multiple such values exist, print all of them, each on a new line.

- If none exist, print `NO SPECIAL VALUE`.

Example

Input:

```
3
1 2 3
4 5 6
7 8 9
```

Output:

```
7
```

Constraints

1. $2 \leq n \leq 20$.
2. Matrix values can be positive, negative, or zero.
3. At most, all rows/columns may or may not contain such a special value.

Note: This problem is an improvisation of question Basic-Array OPS (LAB-5)

#	INPUT	OUTPUT	VISIBILITY
1	3 3 1 3 4 2 6 9 8 7	7	visible
2	3 1 2 3 2 3 4 3 4 5	3	visible
3	3 10 20 30 5 25 35 15 10 5	NO SPECIAL VALUE	visible
4	3 -3 -2 -1 -6 -5 -4 -9 -8 -7	-3	visible
5	3 9 8 7 6 5 4 3 2 1	7	visible
6	3 5 5 5 5 5 5 5 5 5	5 5 5	invisible
7	4 1 2 3 4 9 8 7 6 5 15 25 35 40 30 20 10	NO SPECIAL VALUE	invisible
8	2 10 20 5 25	10	invisible
9	3 -1 0 1 -5 -2 -3 4 3 2	2	invisible
10	1 7	7	invisible
11	2 1 2 3 4	3	invisible

Figure 3: Test-Cases for problem-3

12	2 5 9 7 6	NO SPECIAL VALUE	invisible
13	4 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	13	invisible
14	4 20 30 40 50 10 25 35 45 5 15 25 35 1 2 3 4	20	invisible
15	4 8 7 6 5 12 11 10 9 16 15 14 13 20 19 18 17	17	invisible
16	3 -1 -2 -3 -4 -5 -6 -7 -8 -9	-3	invisible
17	3 0 -2 5 7 3 1 -1 -3 2	NO SPECIAL VALUE	invisible
18	3 0 1 -5 7 3 -2 -1 -3 -10	-2	invisible
19	5 9 8 7 6 5 4 9 8 7 6 3 4 9 8 7 2 3 4 9 8 1 2 3 4 9	NO SPECIAL VALUE	invisible
20	5 1 2 3 4 5 5 4 3 2 1 6 7 8 9 10 10 9 8 7 6 11 12 13 14 15	11	invisible

Figure 4: Test-Cases for problem-3

21	3 1 2 3 4 5 6 7 8 9	7	invisible

Figure 5: Test-Cases for problem-3

```c
#include <stdio.h>

#define MAX 20

// Function to check if a given element is the maximum in its column
int isMaxInCol(int matrix[MAX][MAX], int n, int row, int col) {
    for (int i = 0; i < n; i++) {
        if (matrix[i][col] > matrix[row][col]) {
            return 0; // not maximum in column
        }
    }
    return 1;
}

int main() {
    int n, matrix[MAX][MAX];
    scanf("%d", &n);

    // Input matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    int found = 0;
```

```
27
28      // For each row, check if there's a saddle point
29      for (int i = 0; i < n; i++) {
30          // Find min element in row i
31          int minVal = matrix[i][0], colIndex = 0;
32          for (int j = 1; j < n; j++) {
33              if (matrix[i][j] < minVal) {
34                  minVal = matrix[i][j];
35                  colIndex = j;
36              }
37          }
38
39          // Check if this min element is also the maximum in its column
40          if (isMaxInCol(matrix, n, i, colIndex)) {
41              printf("%d\n", minVal);
42              found = 1;
43          }
44      }
45
46      if (!found) {
47          printf("NO SPECIAL VALUE");
48      }
49
50      return 0;
51  }
```